6. Semafori

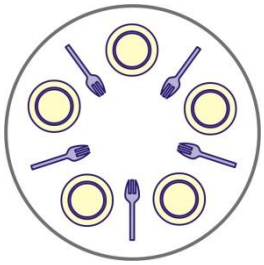
Piemērs

Mērķi: Iemācīties risināt savstarpējas izslēgšanas problēmas, izmantojot semaforus.

Uzdevums: Uzrakstīt programmu, kura risina uzdevumu par pusdienotāja filozofiem. Nepieciešams aprakstīt filozofa dzīves procesu programmas veidā un atrisināt savstarpējās izslēgšanas, kuras var rodas dakšiņu un šķīvju lietošanas laikā, izmantojot semaforus.

6.1. Uzdevums par pusdienotājiem filozofiem

Vienā mājā dzīvo pieci filozofi. To dzīve paiet pārdomās, kuras pārtrauc ar ēdienreizes. Izbadējies filozofs ieiet ēdamistabā, apsēžas pie apaļā galda, paņem divas dakšiņas un ķeras pie azaida. Uz galda ir novietoti pieci šķīvji un starp tiem piecas dakšiņas. Filozofs var paņemt tikai divas tuvākās dakšiņas. Pabeidzot pusdienot, filozofs noliek vietā dakšiņas, iziet no ēdamistabas un turpina pārdomas.



6.2. Uzdevuma risināšanas algoritms

Uzdevuma risināšanas laikā ir jāizslēdz iespēja izmantot vienu dakšiņu ar diviem filozofiem vienlaicīgi. T.i. katrai dakšiņai nepieciešams uzbūvēt kritisko sekciju. Katram filozofam pirms ēdiena sākuma ir jāņem divas dakšiņas t.i. jāienāk uz divās kritiskas sekcijas. Tad uzdevums būs risināts, bet rodas iespēja nobloķēt programmas darbu (savstarpēja bloķēšana). Ja visi filozofi vienlaicīgi ienāk uz ēdamistabu un katrs vienlaicīgi ņem vienu dakšiņu, otro dakšiņu ņemt nesanāks ne kam un visi nomirs no bada.

Lai izslēgtu savstarpējas bloķēšanas iespēju var būt izmantots viena no sekojošiem pieejām:

1. neielaist piecus filozofus vienlaicīgi uz ēdamistabu,
   vai
2. viena no filozofiem var izdarīt ar kreili (t.i. visi filozofi pirmā ņem labēju dakšiņu, bet viens – kreisu).

Dakšiņas kritiskās sekcijas realizācijai var izmantot bināro semaforu (semaforu ar sākumvērtību viens) vai savstarpēju izslēgšanu semaforu (`mutex`).

Filozofu daudzuma, kuri var vienlaicīgi ienākt ēdamistabā, ierobežošanai var izmantot semaforu ar sākumvērtību 4.

6.3. Semafori

Semafors – sinhronizācijas objekts, kas ierobežo pavedienu daudzumu, kuri var iekļūt uz noteikto koda gabalu (jeb uz kritisko sekciju). Semaforu būvē uz vesela skaitļa bāzes (semafora vērtība). Semafors var atrasties vienā no diviem stāvokļiem – signālā ā vai ne signālā stāvoklī. Semafors atrodas signālā stāvoklī ja tā vērtība ir lielāka par nulli, bet ne signālā – ja vienāda ar nulli.

```
Semaphore = 0 → not signal
Semaphore > 0 → signal
```

Ar semaforu tiek definētas vismaz trīs operācijas:

Inicializācija Izveidošanas laikā semaforam ir jāpiešķir nenegatīva sākumvērtība. Windows sistēmā semafora izveidošanai un sākumvērtības piešķiršanai izmanto funkciju `CreateSemaphore`.

Gaidīšana Pirms kritiskas sekcijas ir jāgaida semafora signāla stāvoklis. Ja semafors atrodas signālā stāvoklī, tā vērtība samazinās par vienu un pavediens turpina darbu, citādi pavediens bloķējas gaidīšanas rindā. Windows sistēmā semafora gaidīšanai var izmantot funkciju `WaitForSingleObject`.

Atbrīvošana Pēc kritiskas sekcijas ir jāatbrīvo semafors. Ja atbrīvošanas laikā semafora rindā atrodas pavedieni, viens no tiem atbrīvojas no rindas un ienāk uz kritisko sekciju. Ja semafora rinda ir tukša, semafora vērtība palielinās par vienu. Windows sistēmā semafora atbrīvošanai izmanto funkciju `ReleaseSemaphore`.

6.4. Semafora izveidošana

Windows

```
HANDLE WINAPI CreateSemaphore(
  _In_opt_  LPSECURITY_ATTRIBUTES lpSemaphoreAttributes,
  _In_      LONG lInitialCount,
  _In_      LONG lMaximumCount,
  _In_opt_  LPCTSTR lpName
);
```

`lpSemaphoreAttributes`	Nosaka vai izveidota semafora turis var būt izmantots citos procesos. Ja šis parametrs ir `NULL`, turis nevar būt mantots.
`lInitialCount`	Semafora sākumvērtība.
`lMaximumCount`	Semafora maksimāli iespējama vērtība.
`lpName`	Semafora vārds. Semafora vārds ir jānorāda ja semaforu tiek plānots izmantot dažos procesos. Tad pārēji procesi var griezties pie semafora izmantojot tā vārdu. Ja semafors tiek izmantots tikai vienā procesā, tad pavedieni var griezties pie semafora, norādot tā turu. Šajā gadījumā vārda vietā ir jānorāda `NULL`.
Funkcijas rezultāts	Izveidota semafora turis.

6.5. Semafora gaidīšana

Windows

```
DWORD WINAPI WaitForSingleObject(
  _In_  HANDLE hHandle,
  _In_  DWORD dwMilliseconds
);
```

`hHandle`	Semafora turis.
`dwMilliseconds`	Maksimālais gaidīšanas laiks milisekundēs vai konstante `INFINITE` ja gaidīšanas laiks nav ierobežots.
Funkcijas rezultāts	Gaidīšanas pabeigšanas iemesls: `WAIT_OBJECT_0` semafors pārgāja signālā stāvoklī, `WAIT_TIMEOUT` nobeidzās gaidīšanas laiks

6.6. Semafora atbrīvošana

Windows

```
BOOL WINAPI ReleaseSemaphore(
  _In_       HANDLE hSemaphore,
  _In_       LONG lReleaseCount,
  _Out_opt_  LPLONG lpPreviousCount
);
```

`hSemaphore`	Semafora turis.
`lReleaseCount`	Skaitlis, uz kuru jābūt palielināta semafora vērtība. Parasti semaforu jāpalielina par `1`.
`lpPreviousCount`	Rādītājs uz mainīga lai saņemtu iepriekšējo semafora vērtību. Šis rādītājs var būt `NULL`, ja iepriekšējo vērtību mums zināt nav interesanti.
Funkcijas rezultāts	`TRUE` ja semafora vērtība ir veiksmīgi palielināta vai `FALSE` kļūdas gadījumā.

6.7. Filozofa klase

Programmas uzrakstīšanas laikā filozofu priekšstatam ir jāizmanto klase `TPhilosopher`. Modulis ar šo klasi ir pievienots uzdevumam un tas var būt lejupielādēts no df.rtu.lv elektroniskas bibliotēkas.

Katrs filozofa objekts ir raksturojams ar tekoša stāvokli (ēd, domā, neko nedara vai nomira), ar dzīves spēku, ar gudrību un ar rijību. Uzreiz pēc filozofa izveidošanas iekšpus objekta sāk strādāt pavediens, kurš 10 reizes sekundē izmaina filozofa dzīves spēku un gudrību atkarībā no tā stāvokli un rijības. Kad dzīves spēks sasniegs nulli, filozofs nomirs. Filozofa stāvokļi var izmainīt ar metodēm `Eat` (pārveido uz stāvokli "ēd"), `Think` (pārveido uz stāvokli "domā") un `Kill` (pārveido uz stāvokli "nomira").

`TPhilosopher` klases atklātas metodes:

`void Think();`	Pēc šīs metodes izsaukšanas filozofs sāk domāt un bloķē pavedienu kamēr neizbadēsies. Filozofs atrodas domāšanas stāvoklī nejauša laika intervālā (0.5s – 1s).
`void Eat();`	Filozofs sāk ēst un bloķē pavedienu līdz ēdiena nobeigumam. Filozofs atrodas ēdināšanas stāvoklī nejauša laika intervālā (0.5s – 1s).
`void Kill();`	Nogalināt filozofu.
`int GetState();`	Uzzināt filozofa tekošo stāvokli: `psIdle` filozofs neko nedara (atrodas ārpus `Think` un `Eat` metožu), šajā stāvoklī filozofa dzīves spēks samazinās par diviem punktiem katras 100 milisekundes; `psThink` filozofs domā (atrodas iekšpus `Think` metodes), šajā stāvoklī katras 100 milisekundes filozofa dzīves spēks samazinās par vienu punktu bet gudrība palielinās par vienu punktu; `psEat` filozofs ēd (atrodas iekšpus `Eat` metodes) šajā stāvoklī katras 100 milisekundes filozofa dzīves spēks palielinās par trijiem punktiem; `psDie` filozofs nomira, viņš nomirst kad dzīves spēks samazinās līdz nullei vai pēc `Kill` metodes izsaukšanas.
`int GetForce();`	Atgriež filozofa tekošo dzīves spēku.
`int GetWisdom();`	Atgriež filozofa tekošo gudrību.
`int GetVoracity();`	Atgriež filozofa rijību procentos: 0 - neēd tikai domā, 100 – nedomā tikai ēd.
`bool SetVoracity(int);`	Uzstāda filozofa rijību procentos: 0 - neēd tikai domā, 100 – nedomā tikai ēd.

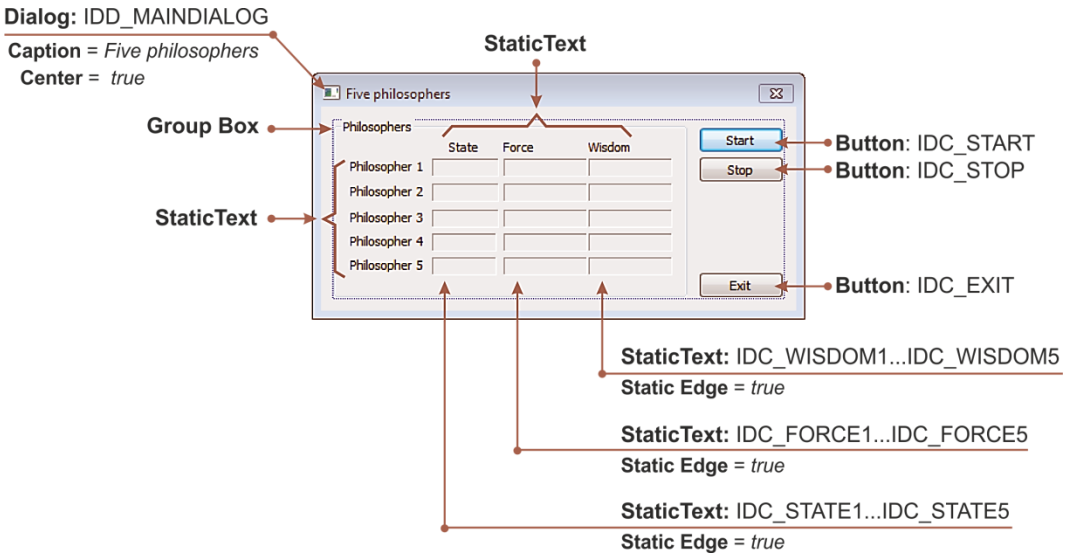
Piemēram, filozofa dzīves kārtība var būt attēlota sekojoši:

```
TPhilosopher * Philosopher = new TPhilosopher;
while (Philosopher->GetState() != psDie) {
      Philosopher->Think();
      Philosopher->Eat();
}
delete Philosopher;
```

6.8. Programmas uzrakstīšanas kārtība

Programmā katram no pieciem filozofiem ir jāizveido filozofa klases objekts un jāuzraksta pavediens, kurš apraksta filozofa dzīves kārtību. Pavedienā ir secīgi jāizsauc filozofa metodes `Eat` (lai filozofs paēstu) un `Think` (lai filozofs padomātu). Pie tam nepieciešams izslēgt vienlaicīgas dakšiņu izmantošanas iespēju un izslēgt savstarpējas bloķēšanas iespēju. Lai būtu iespēja attēlot filozofu stāvokļus uz loga, filozofa objekts katrreiz, kad tā stāvoklis izmainīsies, nosūtīs galvenajam logam paziņojumu `WM_PHOLOSOPHER` ar filozofa numuru (`wParam`) un ar filozofa objekta adresi (`lParam`). Galvenajai formai, reaģējot uz šos paziņojumus, ir jāatjauno informācija par noradīto filozofu uz ekrāna.

1. Galvenās formas šablons



2. Pieslēgt projektam filozofa klasi

1. lejupielādēt arhīvu ar filozofa klases moduli no df.rtu.lv elektroniskas bibliotēkas.
2. Nokopēt failus `Philosopher.lib`[1] un `Philosopher.h` no arhīva uz projekta mapi.
3. `MS Visual Studio` vidē pievienot projektam moduļa failus:
   - izvēlēt izvēlnes punktu `Project | Add Existing Item` vai uzspiest taustiņu kombināciju `Alt+Shift+A`,
   - izvēlēt failus `Philosopher.obj` un `Philosopher.lib` un uzspiest spiedpogu `Add`.
4. Pieslēgt galvenajam modulim filozofa moduļa galvenes failu - `#include "Philosopher.h"`

3. Globālie mainīgie

Deklarēt globālos mainīgos filozofu un semaforu glabāšanai:

- filozofu dinamisko objektu masīvs,
- dakšiņu semaforu turu masīvs,
- ēdamistabas semafora turis.

[1] Filozofa klases bibliotēka var būt izmantota tikai atkļūdošanas (`Debug`) projekta versijas kompilācijai.

4. Filozofa dzīves pavediens

1. Izlasīt filozofa numuru `id` no pavediena parametra.
2. Galvenajā ciklā, kamēr filozofs ar numuru `id` dzīvs:
   - dot filozofam padomāt;
   - ieiet ēdamistabā (pārbaudīt ēdamistabas semaforu);
   - ņemt kreiso dakšiņu (pārbaudīt dakšiņas semaforu ar numuru `id`);
   - ņemt labo dakšiņu (pārbaudīt dakšiņas semaforu ar numuru `(id+1)%5`);
   - dot filozofam paēst;
   - nolikt labo dakšiņu (atbrīvot dakšiņas semaforu ar numuru `(id+1)%5`);
   - nolikt kreiso dakšiņu (atbrīvot dakšiņas semaforu ar numuru `id`);
   - iziet no ēdamistabas (atbrīvot ēdamistabas semaforu).

5. Galvenā loga inicializācija (`WM_INITDIALOG` ziņojums)

1. Izveidot binārus semaforus dakšiņām.
2. Izveidot semaforu ēdamistabai.
3. Uz visās filozofu masīva pozīcijas ierakstīt `NULL`.

6. Galvenā loga nīcināšana (`WM_DESTROY` ziņojums)

1. Katram izveidotam filozofam no globālajā masīva:
   - nogalināt ar komandu `Kill`,
   - dot laiku filozofa dzīves pavedienam pabeigt darbu,
   - dzēst filozofa objektu.
2. Aizvērt dakšiņu semaforu turus.
3. Aizvērt ēdamistabas semafora turu.

7. `Start` spiedpoga

1. Izveidot filozofu objektus un ierakstīt rādītājus uz globālā masīva.
2. Katram filozofam izveidot pavedienu, kā parametru norādīt filozofa numuru, aizvērt pavediena turu.

8. `Stop` spiedpoga

Katram filozofam no masīva:

- nogalināt ar komandu `Kill`,
- dot laiku filozofa dzīves pavedienam pabeigt darbu,
- dzēst filozofa objektu un ierakstīt uz masīva pozīciju `NULL`.

9. Paziņojums no filozofa (`WM_PHILOSOPHER` ziņojums)

1. Noteikt filozofa numuru, kas nosuta paziņojumu (filozofa numurs būs nosūtīts caur `wParam`).
2. Ja filozofa objekts eksistē (rādītājs masīvā nav vienāds `NULL`), izlasīt no tā filozofa stāvokli, dzīves spēku un gudrību un izvadīt tos uz attiecīgus laukus uz ekrāna.
3. Ja filozofa objekts neeksistē (rādītājs masīvā ir vienāds `NULL`), attīrīt attiecīgus laukus uz ekrāna.

6.9. Funkciju saraksts

Šajā darbā vajag iemācīties izmantot sekojošas funkcijas:

`CreateSemaphore`	izveidot semaforu
`ReleaseSemaphore`	atbrīvot semaforu
`CreateMutex`	izveidot savstarpējas izslēgšanas semaforu
`ReleaseMutex`	atbrīvot savstarpējas izslēgšanas semaforu
`WaitForSingleObject`	gaidīt viena objekta signāla stāvokli
`WaitForMultipleObjects`	gaidīt dažu objektu signāla stāvokļus

6.10. Papilduzdevumi

1. Mainīt IDC_START un IDC_STOP spiedpogu stāvokļus simulācijas sākumā un beigās

Izsaucot funkciju `EnableWindow` loga inicializācijas un spiedpogas `IDC_START` un `IDC_STOP` uzspiešanas laikā, iegūt nepieciešamus spiedpogu stāvokļus.

2. Dinamiski mainīt filozofa stāvokļa teksta krāsu

Vadības elementa teksta krāsas maiņai jāuzraksta reakcija uz paziņoju `WM_CTLCOLORSTATIC`, kuru vadības elements sūtīs dialoga logam pirms tas tiks uzzīmēts uz ekrāna. Šā paziņojuma apstrādāšanas laikā var izmainīt zīmēšanas konteksta `DC` parametrus tai skaitā teksta krāsu ar funkciju `SetTextColor`. Piemēru var atrast MSDN sistēmā.

Lai uzzinātu vadības elementu, kas nosūta paziņojumu, ir jāpārbauda ziņojuma parametrs `lParam`. Šīs parametrs saturēs vadības elementa turu. Lai pārveidotu turu uz vadības elementa identifikatoru `ID`, ar kuru būs ērtāk strādāt, var izmantot funkciju `GetDlgCtrlID`.

3. Atļaut lietotājam mainīt filozofu rijību ar papilda dialoga palīdzību

Filozofa rijība iespaido laika sadalīšanu starp ēdienu un domāšanu. Rijība 0 noteic, ka filozofs neēd tikai domā, bet rijība 100 – nedomā tikai ēd. Tekošo rijību var uzzināt ar filozofa metodi `GetVoracity` un izmainīt ar metodi `SetVoracity`.

Lai lietotājam būtu iespēja izmainīt filozofu rijību vajag izveidot jauno dialoga logu, ierakstīt turp tekošo filozofa rijību, ļaut lietotājam rediģēt to un, pēc dialoga aizvēršanas ar spiedpogu `Ok`, izlasīt jauno vērtību no dialoga lauka un ierakstīt uz filozofu objektus. Lai to izdarītu nepieciešams:

1. Izveidot jaunā dialoga loga šablonu, izvietot uz tā vadības elementu, ar kuras palīdzību lietotājs varētu ievadīt veselo skaitli (piem. `Edit Control` vai `Slider Control` jeb `Trackbar`), un spiedpogas `OK` un `CANCEL`.
2. Uzrakstīt dialoga loga ziņojumu apstrādāšanas funkciju, kurai jābūt tādi paši parametri un struktūra kā galvenā loga ziņojumu apstrādāšanas funkcijai, bet cits nosaukums.
3. Ziņojumu apstrādāšanas funkcijā uzrakstīt sekojošu ziņojumu apstrādāšanu:
   - uz ziņojumu `WM_INITDIALOG`: izlasīt rijību no pirmā filozofa un ievietot tā vērtību uz vadības elementa;
   - uz spiedpogas `OK` uzspiešanu: izlasīt vērtību no vadības elementa, ierakstīt to kā filozofa rijību visiem filozofiem un pabeigt dialogu ar funkciju `EndDialog`;
   - uz spiedpogas `CANCEL` uzspiešanu: pabeigt dialogu ar funkciju `EndDialog`.
4. Uz galvenā loga izvietot papildu spiedpogu. Šīs spiedpogas uzspiešanas laikā izveidot jauno dialogu ar funkciju `DialogBox`, kurai ir jānorāda:

`hInstance`	tekošas lietotnes eksemplāra turis no `WinMain` parametriem (lai būtu iespēja šeit izmantot šo turu, funkcijā `WinMain` tas ir jāsaglabā uz globālā mainīgā);
`lpTemplate`	identifikators, ko programmētājs piešķīra dialoga loga šablonam;
`hWndParent`	vecāku loga turis (dialoga logam tas ir galvenā loga turis);
`lpDialogFunc`	dialoga loga ziņojumu apstrādāšanas funkcijas adrese.